

**INSTITUTO FEDERAL DO ESPÍRITO SANTO**

Curso de Engenharia Elétrica

**Daniel Prezotti Marchesi**

## **RELATÓRIO TÉCNICO INDIVIDUAL**

**Projeto VM3 Discovery — Agente de Descoberta de Hosts**

Disciplina: Laboratório de Redes

Guarapari

2026

## 1 INTRODUÇÃO

Este relatório apresenta o desenvolvimento do projeto acadêmico VM3 Discovery, realizado na disciplina de Laboratório de Redes. O projeto consiste na criação de uma máquina virtual responsável por atuar como agente de descoberta e monitoramento de rede. Essa máquina, denominada VM3, executa um scanner capaz de identificar hosts ativos, portas abertas, serviços disponíveis, possíveis sistemas operacionais e mudanças na rede ao longo do tempo.

A proposta do projeto é simular uma solução de inventário e monitoramento semelhante às utilizadas em ambientes corporativos. Em uma rede real, é importante saber quais dispositivos estão ativos, quais serviços estão expostos e quando ocorrem alterações. Por exemplo, a entrada de um novo host, a abertura de uma nova porta ou a indisponibilidade de uma máquina podem indicar mudanças administrativas, falhas ou possíveis riscos de segurança.

Dentro da arquitetura geral do trabalho, a VM3 não funciona de forma isolada. Ela atua como agente coletor, enquanto a VM1 funciona como servidor central. A VM3 realiza as varreduras na rede, organiza os dados em formato JSON e envia essas informações para a VM1 por meio de requisições HTTP. A VM1 é responsável por receber, armazenar e disponibilizar os inventários e eventos recebidos.

Para o desenvolvimento, foram utilizados Ubuntu Server, Python, Nmap, Docker, Docker Compose, GitHub e Docker Hub. A utilização dessas ferramentas permitiu que o scanner fosse desenvolvido, versionado, containerizado e disponibilizado para execução em outras máquinas.

## 2 OBJETIVOS

O objetivo principal do projeto foi desenvolver um agente de descoberta e monitoramento de rede capaz de executar varreduras automáticas em diferentes VLANs do ambiente. Esse agente deveria coletar informações técnicas dos hosts encontrados e enviar os resultados para um servidor central.

Durante o desenvolvimento, buscou-se implementar uma solução capaz de descobrir hosts ativos, identificar portas abertas, reconhecer serviços em execução, realizar fingerprint do sistema operacional e gerar um inventário estruturado. Além disso, o sistema deveria manter um estado anterior da rede para detectar alterações entre uma varredura e outra.

Outro objetivo importante foi tornar o projeto reproduzível. Para isso, o código foi armazenado em um repositório Git e o scanner foi preparado para execução em Docker, de forma que outro usuário possa clonar o projeto, baixar a imagem Docker e executar a VM3 Discovery com o mínimo de etapas manuais.

### **3 ARQUITETURA DO PROJETO**

A arquitetura do projeto foi organizada em duas funções principais: o servidor central, representado pela VM1, e o agente de descoberta, representado pela VM3.

A VM1 atua como ponto central do sistema. Ela recebe os inventários e eventos enviados pela VM3 por meio de uma API REST. Os endpoints utilizados no projeto são:

```
http://10.0.20.10/api/v1/inventory/  
http://10.0.20.10/api/v1/events/
```

O primeiro endpoint é utilizado para envio dos inventários, enquanto o segundo é utilizado para envio de eventos. O inventário representa o estado atual dos hosts encontrados na rede. Já os eventos representam mudanças detectadas pelo scanner, como descoberta de novo host, host indisponível ou nova porta aberta.

A VM3 atua como agente de descoberta. Ela executa o scanner desenvolvido em Python, utiliza o Nmap para realizar as varreduras, organiza os dados em JSON e envia os resultados para a VM1. A VM3 foi configurada com duas interfaces de rede: uma para acesso externo e outra para acesso à rede interna do projeto.

A separação entre VM1 e VM3 é importante porque divide as responsabilidades. A VM3 fica responsável pela coleta das informações, enquanto a VM1 fica responsável pelo recebimento e armazenamento dos dados. Essa organização se aproxima de modelos reais de monitoramento, nos quais agentes distribuídos coletam informações e as enviam para uma central.

### **4 METODOLOGIA, CONFIGURAÇÃO E IMPLEMENTAÇÃO**

A implementação do projeto começou pela criação da máquina virtual no OpenStack. A VM foi criada com Ubuntu Server e recebeu o nome VM3-Descoberta. Após a criação, foi necessário configurar corretamente as interfaces de rede, pois o funcionamento do scanner depende diretamente da conectividade com as VLANs internas.

|                          |                                     |
|--------------------------|-------------------------------------|
| <b>Name</b>              | VM3- Desciberta                     |
| <b>ID</b>                | b318abec-1a41-441f-8cad-8fffb1aff09 |
| <b>Description</b>       | -                                   |
| <b>Project ID</b>        | 90f1c80288444ed4bb4e41b5aa2d003f    |
| <b>Status</b>            | Active                              |
| <b>Locked</b>            | False                               |
| <b>Availability Zone</b> | nova                                |
| <b>Created</b>           | June 22, 2026, 6:27 p.m.            |
| <b>Age</b>               | 3 days, 2 hours                     |
| <b>Host</b>              | server-1                            |
| <b>Instance Name</b>     | instance-00000402                   |
| <b>Reservation ID</b>    | r-wdvz7dll                          |
| <b>Launch Index</b>      | -                                   |
| <b>Hostname</b>          | vm3--desciberta                     |
| <b>Kernel ID</b>         | -                                   |
| <b>Ramdisk ID</b>        | -                                   |
| <b>Device Name</b>       | /dev/vda                            |
| <b>User Data</b>         | -                                   |

|                    |                                      |
|--------------------|--------------------------------------|
| <b>Specs</b>       |                                      |
| <b>Flavor Name</b> | minor.pico.large                     |
| <b>Flavor ID</b>   | ac73a337-a746-4e7c-969f-4c6a3fa889dc |
| <b>RAM</b>         | 1GB                                  |
| <b>VCPUs</b>       | 1 VCPU                               |
| <b>Disk</b>        | 16GB                                 |

|                          |                            |
|--------------------------|----------------------------|
| <b>IP Addresses</b>      |                            |
| <b>labredes1</b>         | 192.168.10.187, 10.10.3.89 |
| <b>VLAN40_DESCOB....</b> | 10.0.40.228                |

|                        |  |
|------------------------|--|
| <b>Security Groups</b> |  |
| Not available          |  |

Figura 1 — Instância VM3-Descoberta criada no OpenStack, apresentando status ativo, recursos da máquina virtual e interfaces de rede associadas.

A Figura 1 evidencia que a instância VM3-Descoberta foi criada no ambiente OpenStack e encontrava-se ativa durante os testes. A tela também permite observar os recursos atribuídos à máquina virtual e os endereços associados às redes do laboratório, servindo como comprovação da infraestrutura utilizada no projeto.

A VM possui duas interfaces principais. A interface ens3 foi utilizada para acesso externo, SSH e comunicação administrativa, recebendo IP da rede externa via DHCP. A interface ens4 foi utilizada para a rede interna do projeto, sendo configurada manualmente com o IP 10.0.40.40/24. A configuração observada foi:

```

root@vm3-descoberta:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:18:87:aa brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.10.187/24 metric 100 brd 192.168.10.255 scope global dynamic ens3
        valid_lft 69780sec preferred_lft 69780sec
    inet6 fe80::f816:3eff:fe18:87aa/64 scope link
        valid_lft forever preferred_lft forever
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:9d:43:e2 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 10.0.40.40/24 brd 10.0.40.255 scope global ens4
        valid_lft forever preferred_lft forever
    inet6 fe80::f816:3eff:fe9d:43e2/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 36:7c:c5:31:05:52 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::347c:c5ff:fe31:552/64 scope link
        valid_lft forever preferred_lft forever
root@vm3-descoberta:~#

```

Figura 2 — Interfaces de rede da VM3, demonstrando a interface ens3 conectada à rede externa e a interface ens4 conectada à VLAN 40.

Na Figura 2, observa-se que a interface ens3 recebeu o endereço 192.168.10.187/24, utilizado para acesso externo e administração da VM por SSH. A interface ens4, por sua vez, recebeu o endereço 10.0.40.40/24, utilizado pelo scanner para acessar a rede interna do projeto. Também aparece a interface docker0, criada automaticamente pelo Docker, mas a execução do scanner utiliza o modo de rede host para trabalhar diretamente com as interfaces da VM.

A configuração da interface interna foi realizada por meio do Netplan, no arquivo `/etc/netplan/50-cloud-init.yaml`. A interface ens3 foi mantida com DHCP, enquanto a ens4 recebeu configuração estática:

```

GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernets:
    ens3:
      dhcp4: true
    ens4:
      match:
        macaddress: "fa:16:3e:9d:43:e2"
      set-name: ens4
      addresses:
        - 10.0.40.40/24
      routes:
        - to: default
          via: 10.0.40.1
      nameservers:
        addresses:
          - 8.8.8.8

```

Figura 3 — Configuração do Netplan utilizada na VM3, definindo ens3 com DHCP e ens4 com IP estático na VLAN 40.

Após editar o arquivo, a configuração foi aplicada com o comando `sudo netplan apply`. Como a VM passou a usar a ens4 como rota padrão, foi necessário ajustar o roteamento para manter o acesso SSH funcionando pela interface externa, evitando problema de roteamento assimétrico. Para isso, foi adicionada uma rota específica para a rede externa pela ens3:

```
ip route replace 10.10.0.0/16 via 192.168.10.1 dev ens3
```

A tabela de rotas resultante apresenta a rota padrão pela ens4 e a rota específica pela ens3:

```

root@vm3-descoberta:~# ip route
default via 10.0.40.1 dev ens4 proto static
10.0.40.0/24 dev ens4 proto kernel scope link src 10.0.40.40
10.10.0.0/16 via 192.168.10.1 dev ens3
169.254.169.254 via 192.168.10.1 dev ens3 proto dhcp src 192.168.10.187 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.10.0/24 dev ens3 proto kernel scope link src 192.168.10.187 metric 100
192.168.10.1 dev ens3 proto dhcp scope link src 192.168.10.187 metric 100

```

Figura 4 — Tabela de rotas da VM3, mostrando a rota padrão pela interface ens4 e a rota específica para a rede externa pela interface ens3.

Para validar o roteamento, foi utilizado o comando `ip route get 8.8.8.8`. O resultado demonstrou que o tráfego para a internet estava saindo pela interface `ens4`, confirmando que a configuração de rota padrão estava correta.

```
root@vm3-descoberta:~# ip route get 8.8.8.8
8.8.8.8 via 10.0.40.1 dev ens4 src 10.0.40.40 uid 0
cache
root@vm3-descoberta:~#
```

Figura 5 — Teste de roteamento mostrando que o tráfego para 8.8.8.8 utiliza a interface `ens4`.

Com a rede configurada, foi implementado o scanner em Python. O código principal está no arquivo `scanner.py`. O scanner utiliza uma lista de redes monitoradas, permitindo percorrer diferentes VLANs. A configuração atual contempla as VLANs 10, 20, 30, 40, 50 e 60:

```
REDES = [
    {"rede": "10.0.10.0/24", "vlan": 10},
    {"rede": "10.0.20.0/24", "vlan": 20},
    {"rede": "10.0.30.0/24", "vlan": 30},
    {"rede": "10.0.40.0/24", "vlan": 40},
    {"rede": "10.0.50.0/24", "vlan": 50},
    {"rede": "10.0.60.0/24", "vlan": 60}
]
```

A descoberta de hosts ativos foi feita com o Nmap utilizando a opção `-sn`, que identifica quais hosts estão ativos sem executar, inicialmente, a varredura de portas. Após identificar os hosts ativos, o scanner executa uma nova etapa para descobrir portas abertas e serviços:

#### 4.1 Descoberta de hosts ativos com Nmap

A descoberta de hosts foi uma das etapas mais importantes do projeto, pois todas as etapas seguintes dependem primeiro de saber quais endereços IP estão realmente ativos. Para isso, o scanner utiliza o Nmap com a opção `-sn`. Essa opção realiza uma varredura de descoberta, também chamada de *host discovery*, sem executar inicialmente a varredura de portas. Assim, o processo fica mais eficiente, pois o scanner evita testar portas em endereços que não responderam na etapa inicial.

```
nmap -sn 10.0.40.0/24
```

Quando esse comando é executado, o Nmap analisa a rede informada e tenta identificar quais hosts respondem. Dependendo da posição da VM3 na rede e das permissões disponíveis, essa descoberta pode envolver requisições ARP para hosts da mesma VLAN, pacotes ICMP e também probes TCP. Na VLAN 40, onde a VM3 está diretamente conectada,

o ARP tende a ser mais eficiente, pois a comunicação ocorre no mesmo domínio de broadcast. Em outras VLANs, a descoberta depende do roteamento e das respostas dos hosts remotos.

No código, essa etapa é implementada na função `descobrir_hosts_rede(rede)`. O Python executa o Nmap usando `subprocess.run()`, captura a saída do comando e percorre linha por linha procurando os trechos que indicam um host encontrado. O Nmap geralmente exibe esses resultados no formato "Nmap scan report for <IP>". O scanner interpreta essas linhas e extrai o endereço IP encontrado.

```
def descobrir_hosts_rede(rede):
    resultado = subprocess.run(
        ["nmap", "-sn", rede],
        capture_output=True,
        text=True
    )

    hosts = []

    for linha in resultado.stdout.splitlines():
        if "Nmap scan report for" in linha:
            ip = linha.split()[-1]
            ip = ip.replace("(", "").replace(")", "")
            hosts.append(ip)

    return sorted(list(set(hosts)))
```

O parâmetro `capture_output=True` faz com que a saída do comando Nmap seja capturada pelo Python em vez de aparecer apenas no terminal. Já `text=True` converte essa saída para texto comum, permitindo que o código use operações como `splitlines()` e procure padrões dentro das linhas. Ao final, a função remove duplicatas com `set()` e retorna a lista ordenada de IPs ativos.

A função `descobrir_hosts()` complementa esse processo percorrendo a lista de redes cadastradas no código. Para cada VLAN, ela chama `descobrir_hosts_rede()` e associa cada IP encontrado ao número da VLAN correspondente. Dessa forma, o inventário não guarda apenas o IP, mas também a rede de origem daquele host.

```
for item in REDES:
    rede = item["rede"]
    vlan = item["vlan"]
    ips = descobrir_hosts_rede(rede)

    for ip in ips:
        hosts.append({
            "ip": ip,
            "vlan": vlan,
            "rede": rede
        })

nmap -Pn -T4 -sV <IP>
```



A opção `-Pn` foi usada porque o host já havia sido identificado como ativo na etapa anterior. A opção `-T4` foi escolhida para acelerar a varredura, e a opção `-sV` foi utilizada para identificar os serviços associados às portas abertas. Além disso, o scanner realiza uma tentativa de identificação do sistema operacional utilizando o fingerprint do Nmap:

## 4.2 Descoberta de portas abertas e serviços

Após a etapa de descoberta de hosts, o scanner passa a analisar individualmente cada IP encontrado. Nessa segunda etapa, o objetivo é descobrir quais portas TCP estão abertas e quais serviços estão respondendo nessas portas. Para isso, é utilizado o comando Nmap com as opções `-Pn`, `-T4`, `--version-light` e `-sV`.

```
nmap -Pn -T4 --version-light -sV <IP>
```

A opção `-Pn` informa ao Nmap que o host deve ser tratado como ativo. Isso evita que o Nmap repita a etapa de descoberta de host, pois o scanner já fez essa verificação anteriormente com `nmap -sn`. A opção `-T4` aumenta a velocidade da varredura, mantendo um equilíbrio aceitável entre rapidez e estabilidade. A opção `--version-light` reduz a intensidade da detecção de versão, tornando o processo mais leve. Por fim, `-sV` ativa a identificação do serviço que está associado a cada porta aberta.

No código, essa lógica aparece na função `descobrir_servicos(ip)`. A função executa o Nmap e interpreta as linhas da saída que seguem o padrão de porta aberta, como `"22/tcp open ssh"` ou `"80/tcp open http"`. Para extrair essas informações, foi usada uma expressão regular que captura o número da porta e o nome do serviço.

```
def descobrir_servicos(ip):
    portas = []
    servicos = []

    resultado = subprocess.run(
        ["nmap", "-Pn", "-T4", "--version-light", "-sV", ip],
        capture_output=True,
        text=True
    )

    for linha in resultado.stdout.splitlines():
        match = re.match(r"^(\d+)/tcp\s+open\s+(\S+)", linha)
        if match:
            porta = int(match.group(1))
            servico = match.group(2)
            portas.append(porta)
            servicos.append(servico)

    return portas, servicos
```

Com esse tratamento, uma linha como "22/tcp open ssh" gera a porta 22 na lista open\_ports e o serviço ssh na lista services. Essas duas listas são armazenadas no inventário de forma relacionada, ou seja, a posição da porta na lista de portas corresponde à posição do serviço na lista de serviços. Isso é importante posteriormente para detectar novas portas abertas e identificar qual serviço está associado a elas.

Essa etapa é relevante porque uma porta aberta indica que existe algum processo escutando conexões naquele host. A identificação do serviço torna o inventário mais útil, pois uma porta isolada não informa completamente o que está exposto. Por exemplo, a porta 22 geralmente indica SSH, a porta 80 indica HTTP e as portas 389 e 636 indicam serviços LDAP e LDAPS.

```
nmap -O --osscan-guess <IP>
```

Esse processo analisa características das respostas da pilha TCP/IP, como TTL, flags, janela TCP e respostas ICMP. Como essa identificação pode não ser totalmente precisa, o resultado é armazenado como uma estimativa. No inventário, aparecem informações como Linux 4.15 (96%) e Linux 5.0 - 5.14 (98%).

### 4.3 Fingerprint de sistema operacional

A tentativa de identificação do sistema operacional foi implementada por meio do fingerprint do Nmap. Diferentemente da identificação de serviço, que observa respostas de aplicações em portas abertas, o fingerprint de sistema operacional observa o comportamento da pilha TCP/IP do host. Sistemas diferentes podem responder de formas ligeiramente diferentes a determinados pacotes, e o Nmap compara essas respostas com uma base de assinaturas conhecidas.

```
nmap -Pn -T4 -O --osscan-guess <IP>
```

A opção -O ativa a detecção de sistema operacional. A opção --osscan-guess permite que o Nmap apresente uma estimativa mesmo quando não existe correspondência perfeita. Por isso, os resultados podem aparecer com porcentagem de confiança, como Linux 4.15 (96%) ou Linux 5.0 - 5.14 (98%). No projeto, foi decidido armazenar a primeira sugestão retornada pelo Nmap para manter o inventário simples e objetivo.

```
def descobrir_so(ip):
    resultado = subprocess.run(
        ["nmap", "-Pn", "-T4", "-O", "--osscan-guess", ip],
```

```

        capture_output=True,
        text=True
    )

    for linha in resultado.stdout.splitlines():
        if "OS details:" in linha:
            return linha.replace("OS details:", "").strip()
        if "Running:" in linha:
            return linha.replace("Running:", "").strip()
        if "Aggressive OS guesses:" in linha:
            guess = linha.replace("Aggressive OS guesses:", "").strip()
            return guess.split(",")[0].strip()

    return "Desconhecido"

```

O código foi preparado para lidar com diferentes formatos de saída do Nmap. Em alguns casos, o Nmap retorna a linha OS details. Em outros, retorna Running. Também pode retornar Aggressive OS guesses, contendo várias possibilidades separadas por vírgula. Quando nenhuma dessas informações aparece, o scanner registra o campo como Desconhecido. Esse comportamento é esperado quando o host possui firewall, responde poucos probes, não possui portas suficientes abertas ou fechadas, ou quando o Nmap não consegue obter dados suficientes para uma estimativa confiável.

Também foi implementada a tentativa de descoberta de hostname e endereço MAC dos hosts encontrados. Essas duas informações tornam o inventário mais completo, pois o endereço IP identifica o host na camada de rede, enquanto o hostname facilita a leitura humana do inventário. Já o endereço MAC identifica a interface de rede no nível de enlace, sendo útil principalmente para diferenciar dispositivos dentro da mesma VLAN.

A descoberta de hostname foi realizada inicialmente por meio de resolução reversa de DNS, utilizando a função `socket.gethostbyaddr(ip)` no Python. Essa função tenta consultar se existe algum nome associado ao endereço IP informado. Quando há DNS reverso configurado, o scanner consegue retornar um nome, como ocorreu com a própria VM3, que apareceu no inventário como `vm3-descoberta`.

Entretanto, os hosts da rede possuem registro DNS reverso configurado. Quando isso acontece, a função não consegue retornar um nome válido. Para evitar que o campo fique vazio, foi adotado um tratamento simples: quando o hostname não é encontrado, o próprio endereço IP é utilizado como identificação. Por esse motivo, alguns hosts aparecem no inventário com o campo hostname igual ao IP, como `10.0.10.10`, `10.0.20.1` ou `10.0.60.10`. Isso não significa falha no scanner, mas sim ausência de registro DNS reverso para esses endereços.

Como melhoria futura para esse ponto, recomenda-se configurar DNS reverso no servidor de nomes do ambiente, criando registros PTR para os endereços das VLANs monitoradas. Dessa forma, a identificação de hostname deixaria de depender apenas de respostas ocasionais e passaria a ser padronizada pela infraestrutura de DNS. Essa melhoria não foi finalizada dentro do prazo do projeto, por isso o scanner manteve o comportamento seguro de usar o próprio IP quando nenhum nome é retornado.

A descoberta do endereço MAC foi realizada utilizando informações locais da interface de rede e a tabela de vizinhança do sistema Linux, acessada por meio do comando `ip neigh`. Esse comando consulta a tabela ARP/neighbor da máquina e permite identificar o endereço MAC de hosts que estão no mesmo domínio de broadcast da VM3.

No entanto, existe uma limitação importante: o protocolo ARP opera na camada 2 do modelo de redes e não atravessa roteadores. Como a VM3 está conectada diretamente à VLAN 40, ela consegue identificar melhor os endereços MAC dos hosts que também estão nessa VLAN. Para hosts em outras VLANs, como VLAN 10, VLAN 20, VLAN 50 e VLAN 60, o tráfego passa pelo gateway 10.0.40.1. Nesse caso, a VM3 não se comunica diretamente em camada 2 com o host remoto, mas sim com o roteador intermediário. Por isso, o MAC real desses hosts pode não aparecer no inventário.

Essa limitação pode ser observada nos resultados gerados. Na VLAN 40, alguns hosts aparecem com endereço MAC identificado, como a própria VM3, que possui o MAC `fa:16:3e:9d:43:e2`. Já em outras VLANs, o campo aparece como Desconhecido. Esse comportamento é esperado em redes roteadas e não representa erro de implementação. Para obter o MAC real de hosts em outras VLANs, seria necessário consultar uma fonte que esteja diretamente conectada a essas redes, como o gateway, o OpenStack ou um agente instalado em cada VLAN.

Neste projeto, foi adotada a solução de registrar o MAC como Desconhecido quando a informação não pôde ser obtida diretamente. Essa decisão evita inserir dados incorretos no inventário e deixa explícito que a informação não estava disponível a partir da perspectiva da VM3.

O scanner gera um inventário em JSON com as informações coletadas. Cada host possui campos como hostname, IP, MAC, portas abertas, fingerprint de sistema operacional, serviços e VLAN. Esse formato foi escolhido por ser compatível com APIs REST e de fácil interpretação.

```

    ],
    "vlan": 10
  },
  {
    "hostname": "10.0.10.10",
    "ip_address": "10.0.10.10",
    "mac_address": "Desconhecido",
    "open_ports": [
      389,
      636
    ],
    "os_fingerprint": "Linux 4.15 - 5.19 (97%)",
    "services": [
      "ldap",
      "ldaps"
    ],
    "vlan": 10
  },
  {
    "hostname": "10.0.20.1",
    "ip_address": "10.0.20.1",
    "mac_address": "Desconhecido",
    "open_ports": [
      22,
      5000
    ],
    "os_fingerprint": "Linux 4.15 (96%)",
    "services": [
      "ssh",
      "http"
    ],
    "vlan": 20
  },
  {
    "hostname": "10.0.20.10",
    "ip_address": "10.0.20.10",
    "mac_address": "Desconhecido",
    "open_ports": [
      80
    ],
    "os_fingerprint": "Linux 5.0 - 5.14 (98%)",
    "services": [
      "http"
    ],
    "vlan": 20
  }
]

```

Figura 6 — Inventário JSON gerado pelo scanner, contendo informações de hosts, portas abertas, serviços, fingerprint de sistema operacional e VLAN.

A Figura 6 demonstra um exemplo prático do inventário gerado pelo scanner. Nela é possível observar que cada host é armazenado como um objeto JSON contendo campos padronizados, como hostname, ip\_address, mac\_address, open\_ports, os\_fingerprint, services e vlan.

Observa-se que alguns hosts aparecem com o campo hostname igual ao próprio endereço IP. Isso ocorre porque não foi encontrado registro DNS reverso para esses endereços. Nesses casos, o scanner utiliza o IP como identificador, garantindo que o campo não fique vazio. Também é possível observar que alguns hosts apresentam o campo mac\_address como Desconhecido. Esse resultado é esperado principalmente para hosts localizados em VLANs diferentes da VLAN 40, pois a VM3 não consegue obter diretamente o MAC de dispositivos que estão atrás de um roteador.

A figura também mostra que o fingerprint de sistema operacional não é uniforme para todos os hosts. Em alguns casos, o Nmap retorna uma estimativa com percentual de

confiança, como Linux 4.15 - 5.19 (97%). Em outros casos, o sistema operacional aparece como Desconhecido. Isso acontece porque a identificação de sistema operacional depende das respostas do host aos pacotes enviados pelo Nmap. Hosts com poucas portas abertas, firewall ativo ou respostas limitadas podem não fornecer informações suficientes para uma identificação confiável.

Dessa forma, a Figura 6 não apenas comprova a geração do inventário, mas também evidencia limitações reais de descoberta em redes roteadas. Essas limitações foram tratadas no projeto por meio do uso de valores padronizados, como Desconhecido, e pela manutenção do IP como identificador quando o hostname não está disponível.

Para transformar a ferramenta em monitoramento contínuo, o scanner foi configurado para executar em loop. Ao final de cada ciclo, ele aguarda 60 segundos antes de iniciar uma nova varredura:

```
while True:
    executar_scan()
    time.sleep(60)
```

Além de gerar o inventário atual, o scanner armazena o estado anterior da rede em arquivo JSON. Esse estado é utilizado para comparar a varredura atual com a anterior, permitindo detectar novos hosts, hosts que deixaram de responder e novas portas abertas. Esses eventos são enviados para a VM1 pelo endpoint de eventos.

#### 4.4 Arquivos `inventory.json` e `estado_rede.json`

O projeto utiliza dois arquivos JSON com finalidades diferentes. O arquivo `inventory.json` representa o inventário do scan atual, ou seja, aquilo que foi encontrado na varredura mais recente. Ele é útil para consulta e registro dos dados coletados naquele ciclo. Já o arquivo `estado_rede.json` funciona como uma memória interna do scanner. Ele armazena o último estado conhecido da rede para permitir comparação com o próximo scan.

Essa separação é importante porque o scanner precisa saber o que mudou. Sem o estado anterior, ele conseguiria apenas listar os hosts encontrados no momento, mas não conseguiria afirmar se um host é novo, se um host deixou de responder ou se uma porta foi aberta depois do scan anterior.

```
def carregar_estado_anterior():
    if not os.path.exists(ARQUIVO_ESTADO):
        return {"hosts": {}}

    try:
```

```

        with open(ARQUIVO_ESTADO, "r") as arquivo:
            return json.load(arquivo)
    except Exception:
        return {"hosts": {}}

```

A função `carregar_estado_anterior()` verifica se o arquivo `estado_rede.json` existe. Se ele ainda não existe, significa que o scanner está executando o primeiro ciclo e, portanto, não há estado anterior para comparar. Nesse caso, o código retorna uma estrutura vazia com `hosts` sem registros. Se o arquivo existe, ele é aberto e carregado com `json.load()`. Caso o arquivo esteja corrompido ou vazio, o tratamento de exceção impede que o scanner pare de funcionar.

```

def salvar_estado_atual(inventario):
    estado = {
        "updated_at": agora_iso(),
        "hosts": {}
    }

    for host in inventario["hosts"]:
        ip = host["ip_address"]
        estado["hosts"][ip] = {
            "hostname": host["hostname"],
            "mac_address": host["mac_address"],
            "open_ports": host["open_ports"],
            "services": host["services"],
            "os_fingerprint": host["os_fingerprint"],
            "vlan": host["vlan"]
        }

    with open(ARQUIVO_ESTADO, "w") as arquivo:
        json.dump(estado, arquivo, indent=4)

```

Ao salvar o estado atual, o IP de cada host é usado como chave do dicionário. Essa decisão facilita a comparação, pois no próximo ciclo o código consegue verificar rapidamente se determinado IP já existia antes. O arquivo `estado_rede.json`, portanto, não é apenas uma cópia do inventário; ele é uma estrutura otimizada para comparar estados da rede.

#### 4.5 Identificação de novos hosts, host down e novas portas abertas

A comparação entre o estado anterior e o inventário atual é feita na função `analisar_mudancas()`. Essa função transforma as listas de hosts em conjuntos de IPs. O uso de conjuntos facilita operações matemáticas de diferença, permitindo descobrir quais IPs apareceram e quais desapareceram entre uma varredura e outra.

```

hosts_anteriores = set(
    estado_anterior.get("hosts", {}).keys()
)

hosts_atuais = set(
    host["ip_address"] for host in inventario_atual["hosts"]
)

```

```
)

novos_hosts = sorted(list(hosts_atuais - hosts_anteriores))
hosts_down = sorted(list(hosts_anteriores - hosts_atuais))
```

A expressão `hosts_atuais - hosts_anteriores` retorna os IPs que existem no scan atual, mas não existiam no scan anterior. Esses IPs são classificados como novos hosts. Já a expressão `hosts_anteriores - hosts_atuais` retorna os IPs que existiam antes, mas não aparecem mais no scan atual. Esses IPs são classificados como host down, indicando que deixaram de responder ou saíram da rede.

Para a detecção de novas portas, o scanner compara as portas abertas de um host que já existia anteriormente. Se uma porta aparece na lista atual, mas não estava na lista anterior, ela é considerada uma nova porta aberta. Esse evento é importante porque pode indicar que um novo serviço foi iniciado em uma máquina ou que houve alteração na exposição da rede.

```
portas_anteriores = set(
    estado_anterior["hosts"][ip].get("open_ports", [])
)

portas_atuais = set(
    host.get("open_ports", [])
)

novas_portas = sorted(
    list(portas_atuais - portas_anteriores)
)
```

Quando uma nova porta é identificada, o código também tenta associar essa porta ao serviço correspondente. Isso é feito porque as listas `open_ports` e `services` são montadas na mesma ordem. Assim, se a porta 80 aparece na posição 1 da lista de portas, o serviço na posição 1 da lista de serviços tende a ser o HTTP. Com isso, o evento enviado para a VM1 fica mais informativo.

```
if porta in host["open_ports"]:
    indice = host["open_ports"].index(porta)

    if indice < len(host["services"]):
        servico = host["services"][indice]
```

#### 4.6 Estrutura dos eventos enviados para a VM1

Os eventos foram padronizados para que a VM1 consiga interpretar as mudanças de forma organizada. Cada evento possui uma descrição, um tipo, dados extras, severidade e origem. Essa estrutura facilita o armazenamento e a exibição posterior dos alertas.

```
def evento_novo_host(ip, vlan):
    return {
        "description": f"Novo host detectado na VLAN {vlan}: {ip}",
```



```

        "event_type": "host_discovered",
        "extra_data": {
            "ip": ip,
            "vlan": vlan,
            "detected_at": agora_iso()
        },
        "severity": "info",
        "source": "vm3-descoberta"
    }

```

O evento `host_discovered` é usado quando um IP aparece no scan atual e não existia no estado anterior. Sua severidade foi definida como `info` porque representa uma informação relevante, mas não necessariamente um problema.

```

def evento_host_down(ip, vlan):
    return {
        "description": f"Host {ip} não está mais respondendo na VLAN {vlan}",
        "event_type": "host_down",
        "extra_data": {
            "ip": ip,
            "vlan": vlan,
            "detected_at": agora_iso()
        },
        "severity": "critical",
        "source": "vm3-descoberta"
    }

```

O evento `host_down` é usado quando um host estava presente no estado anterior, mas não aparece no scan atual. A severidade foi definida como `critical` porque a indisponibilidade de um host pode representar falha, desligamento ou perda de comunicação.

```

def evento_nova_porta(ip, vlan, porta, servico):
    return {
        "description": f"Nova porta aberta detectada em {ip}: {porta}/{servico}",
        "event_type": "new_open_port",
        "extra_data": {
            "ip": ip,
            "port": porta,
            "service": servico,
            "vlan": vlan,
            "detected_at": agora_iso()
        },
        "severity": "warning",
        "source": "vm3-descoberta"
    }

```

O evento `new_open_port` é usado quando um host já conhecido passa a apresentar uma porta que não existia anteriormente. A severidade foi definida como `warning` porque uma nova porta aberta pode indicar alteração de configuração, ativação de serviço ou aumento de superfície de exposição da rede.

Por fim, o projeto foi preparado para execução em Docker. A imagem foi publicada no Docker Hub como `danielpmarchesi/vm3labredes:v1`. Para facilitar a execução, foi criado um arquivo `docker-compose.yml` configurado para executar o container com acesso direto à rede da VM por meio de `network_mode: host`, configuração necessária para que o scanner enxergue a rede como se estivesse rodando diretamente no sistema operacional:

```
root@vm3-descoberta:~/vm3-discovery# cat docker-compose.yml
services:
  vm3-discovery:
    build: .
    image: danielpmarchesi/vm3labredes:v1
    container_name: vm3-discovery
    network_mode: host
    privileged: true
    restart: unless-stopped
    volumes:
      - ./app
    environment:
      - PYTHONUNBUFFERED=1
    command: python -u scanner.py
root@vm3-descoberta:~/vm3-discovery#
```

Figura 7 — Arquivo `docker-compose.yml` utilizado para executar o scanner em container Docker.

```
root@vm3-descoberta:~/vm3-discovery# docker images
```

| IMAGE                                       | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|---------------------------------------------|--------------|------------|--------------|-------|
| <code>danielpmarchesi/vm3labredes:v1</code> | 90140ad51ed0 | 253MB      | 60.8MB       | U     |
| <code>hello-world:latest</code>             | 96498ffd522e | 25.9kB     | 9.49kB       | U     |

Figura 9 — Imagem Docker da VM3 Discovery disponível localmente.

## 5 COMANDOS UTILIZADOS

Durante o desenvolvimento do projeto, foram utilizados comandos de rede, Docker e Git. Esses comandos tiveram papel importante tanto na configuração da VM3 quanto na validação do funcionamento do scanner.

Inicialmente, foram utilizados comandos de diagnóstico de rede. O comando `ip` a foi usado para verificar as interfaces disponíveis na máquina virtual. Por meio dele, foi possível confirmar que a interface `ens3` estava associada à rede externa, recebendo o endereço `192.168.10.187/24`, enquanto a interface `ens4` estava configurada na rede interna, com o endereço `10.0.40.40/24`.

`ip a`

O comando `ip route` foi utilizado para verificar a tabela de roteamento da VM3. Esse comando foi essencial porque a VM possui duas interfaces de rede e, por isso, era necessário garantir que cada tipo de tráfego saísse pela interface correta. A rota padrão foi configurada pela interface `ens4`, utilizando o gateway `10.0.40.1`, enquanto a rota para a rede externa `10.10.0.0/16` foi mantida pela interface `ens3`.

```
ip route
sudo ip route replace 10.10.0.0/16 via 192.168.10.1 dev ens3
```

Para validar o caminho usado por determinado tráfego, foi utilizado o comando `ip route get`. O comando `ip route get 8.8.8.8` mostrou que o acesso à internet estava saindo pela interface `ens4`, confirmando a configuração da rota padrão. Já o comando `ip route get 10.0.60.10` foi utilizado para verificar o acesso a outra VLAN, demonstrando que o tráfego para essa rede passava pelo gateway interno.

```
ip route get 8.8.8.8
ip route get 10.0.60.10
```

Também foram utilizados comandos `ping` para validar conectividade. O comando `ping -c 4 8.8.8.8` foi usado para testar acesso externo. O comando `ping -c 4 10.0.40.1` validou a comunicação com o gateway da VLAN 40. Já os comandos `ping -c 4 10.0.20.10` e `ping -c 4 10.0.60.10` foram utilizados para verificar comunicação com outras VLANs do ambiente.

```
ping -c 4 8.8.8.8
ping -c 4 10.0.40.1
ping -c 4 10.0.20.10
ping -c 4 10.0.60.10
```

Na parte de Docker, o comando `docker pull danielpmarchesi/vm3labredes:v1` foi utilizado para baixar a imagem do scanner publicada no Docker Hub. O comando `docker compose up -d` foi utilizado para iniciar o container em segundo plano. O comando `docker ps` permitiu verificar se o container estava em execução. Já o comando `docker images` foi utilizado para confirmar que a imagem estava disponível localmente na VM3.

```
docker pull danielpmarchesi/vm3labredes:v1
docker compose up -d
docker ps
docker images
```

Para acompanhar a execução do scanner, foi utilizado o comando `docker logs -f vm3-discovery`. Esse comando foi um dos mais importantes durante os testes, pois permitiu

visualizar em tempo real o início das varreduras, a descoberta dos hosts, a geração do inventário JSON e o envio dos dados para a VM1. Para interromper a execução do container, quando necessário, foi utilizado o comando `docker compose down`.

```
docker logs -f vm3-discovery
docker compose down
```

Na parte de versionamento, foram utilizados comandos Git para controlar os arquivos do projeto. O comando `git status` permitiu verificar quais arquivos haviam sido modificados. O comando `git add .` foi usado para adicionar os arquivos ao controle de versão. Em seguida, o comando `git commit -m "mensagem do commit"` registrou as alterações no histórico do projeto. Por fim, o comando `git push -u origin main` foi utilizado para enviar os arquivos ao GitHub. O repositório pode ser clonado com `gh repo clone DanielPMarchesi/vm3-discovery`.

```
git status
git add .
git commit -m "mensagem do commit"
git push -u origin main
gh repo clone DanielPMarchesi/vm3-discovery
```

## 6 TESTES EXECUTADOS E RESULTADOS OBTIDOS

Após a configuração e implementação, foram realizados testes para validar o funcionamento da VM3 Discovery. O primeiro teste foi a verificação das interfaces de rede com `ip a`, confirmando que a VM possuía duas interfaces ativas: `ens3` ligada à rede externa e `ens4` ligada à rede interna.

Em seguida, foi verificada a tabela de rotas com `ip route`. O resultado confirmou que a rota padrão estava configurada pela `ens4`, enquanto a rota para a rede `10.10.0.0/16` estava configurada pela `ens3`. Essa separação foi fundamental para manter o acesso SSH funcionando sem prejudicar a comunicação com as VLANs internas.

O comando `ip route get 8.8.8.8` confirmou que o tráfego para a internet saía pela `ens4` usando o gateway `10.0.40.1`, validando que a configuração de rota padrão estava correta.

Depois da validação de rede, foi testada a imagem Docker. O scanner foi executado em container e os logs foram acompanhados. O sistema percorreu as redes `10.0.10.0/24`, `10.0.20.0/24`, `10.0.30.0/24`, `10.0.40.0/24`, `10.0.50.0/24` e `10.0.60.0/24`, demonstrando que a descoberta multi-VLAN estava funcionando.

```
=====
[2026-06-25T21:00:47.801920+00:00] Novo scan iniciado
=====

Descobrindo hosts na rede 10.0.10.0/24 VLAN 10...
Descobrindo hosts na rede 10.0.20.0/24 VLAN 20...
Descobrindo hosts na rede 10.0.30.0/24 VLAN 30...
Descobrindo hosts na rede 10.0.40.0/24 VLAN 40...
Descobrindo hosts na rede 10.0.50.0/24 VLAN 50...
Descobrindo hosts na rede 10.0.60.0/24 VLAN 60...
Escaneando 10.0.10.1 VLAN 10...
Escaneando 10.0.10.10 VLAN 10...
Escaneando 10.0.20.1 VLAN 20...
Escaneando 10.0.20.10 VLAN 20...
Escaneando 10.0.30.1 VLAN 30...
Escaneando 10.0.40.1 VLAN 40...
Escaneando 10.0.40.10 VLAN 40...
Escaneando 10.0.40.20 VLAN 40...
Escaneando 10.0.40.30 VLAN 40...
Escaneando 10.0.40.40 VLAN 40...
Escaneando 10.0.50.1 VLAN 50...
Escaneando 10.0.50.10 VLAN 50...
Escaneando 10.0.60.1 VLAN 60...
Escaneando 10.0.60.10 VLAN 60...
Escaneando 10.0.60.20 VLAN 60...
```

Figura 10 — Scanner iniciando um novo ciclo de varredura nas VLANs 10, 20, 30, 40, 50 e 60.

O inventário gerado apresentou hosts em diferentes VLANs. Foram identificados hosts como 10.0.10.1, 10.0.20.1, 10.0.30.1, 10.0.40.1, 10.0.40.10, 10.0.40.30, 10.0.40.40, 10.0.50.1 e 10.0.60.1, entre outros. Em alguns hosts, foram identificadas portas como 22, 80, 389, 636 e 5000, com serviços como SSH, HTTP, LDAP e LDAPS. Em alguns casos, o fingerprint indicou versões aproximadas de Linux.

```

    ],
    "vlan": 10
  },
  {
    "hostname": "10.0.10.10",
    "ip_address": "10.0.10.10",
    "mac_address": "Desconhecido",
    "open_ports": [
      389,
      636
    ],
    "os_fingerprint": "Linux 4.15 - 5.19 (97%)",
    "services": [
      "ldap",
      "ldapssl?"
    ],
    "vlan": 10
  },
  {
    "hostname": "10.0.20.1",
    "ip_address": "10.0.20.1",
    "mac_address": "Desconhecido",
    "open_ports": [
      22,
      5000
    ],
    "os_fingerprint": "Linux 4.15 (96%)",
    "services": [
      "ssh",
      "http"
    ],
    "vlan": 20
  },
  {
    "hostname": "10.0.20.10",
    "ip_address": "10.0.20.10",
    "mac_address": "Desconhecido",
    "open_ports": [
      80
    ],
    "os_fingerprint": "Linux 5.0 - 5.14 (98%)",
    "services": [
      "http"
    ],
    "vlan": 20
  }
]

```

Figura 11 — Parte do inventário gerado pelo scanner, mostrando hosts encontrados nas VLANs 10 e 20.

```

    ],
    "os_fingerprint": "Linux 4.15 (96%)",
    "services": [
        "ssh"
    ],
    "vlan": 40
},
{
    "hostname": "10.0.40.30",
    "ip_address": "10.0.40.30",
    "mac_address": "fa:16:3e:69:21:1b",
    "open_ports": [
        22
    ],
    "os_fingerprint": "Linux 4.15 (96%)",
    "services": [
        "ssh"
    ],
    "vlan": 40
},
{
    "hostname": "vm3-descoberta",
    "ip_address": "10.0.40.40",
    "mac_address": "fa:16:3e:9d:43:e2",
    "open_ports": [
        22
    ],
    "os_fingerprint": "Linux 2.6.X|5.X",
    "services": [
        "ssh"
    ],
    "vlan": 40
},
{
    "hostname": "10.0.50.1",
    "ip_address": "10.0.50.1",
    "mac_address": "Desconhecido",
    "open_ports": [
        22,
        5000
    ],
    "os_fingerprint": "Linux 4.15 (96%)",
    "services": [
        "ssh",
        "http"
    ]
}

```

Figura 12 — Parte do inventário mostrando hosts da VLAN 40, incluindo a própria VM3.

```
    ],
    "vlan": 50
  },
  {
    "hostname": "10.0.50.10",
    "ip_address": "10.0.50.10",
    "mac_address": "Desconhecido",
    "open_ports": [],
    "os_fingerprint": "Desconhecido",
    "services": [],
    "vlan": 50
  },
  {
    "hostname": "10.0.60.1",
    "ip_address": "10.0.60.1",
    "mac_address": "Desconhecido",
    "open_ports": [
      22,
      5000
    ],
    "os_fingerprint": "Linux 4.15 (96%)",
    "services": [
      "ssh",
      "http"
    ],
    "vlan": 60
  },
  {
    "hostname": "10.0.60.10",
    "ip_address": "10.0.60.10",
    "mac_address": "Desconhecido",
    "open_ports": [],
    "os_fingerprint": "Desconhecido",
    "services": [],
    "vlan": 60
  },
  {
    "hostname": "10.0.60.20",
    "ip_address": "10.0.60.20",
    "mac_address": "Desconhecido",
    "open_ports": [],
    "os_fingerprint": "Desconhecido",
    "services": [],
    "vlan": 60
  }
}
```

Figura 13 — Parte do inventário mostrando hosts encontrados nas VLANs 50 e 60.

A comparação entre as Figuras 11, 12 e 13 demonstra que a obtenção de MAC, hostname e fingerprint depende da posição da VM3 na rede e das respostas fornecidas por



cada host. Na Figura 12, referente à VLAN 40, aparecem hosts com MAC identificado, incluindo a própria VM3. Já nas Figuras 11 e 13, referentes a outras VLANs, é mais comum encontrar MAC como Desconhecido e hostname igual ao IP. Esse resultado está coerente com o funcionamento de redes roteadas, pois a VM3 não está no mesmo domínio de broadcast dos hosts remotos.

Ao final do ciclo de varredura, os logs mostraram que os inventários foram enviados para a VM1 e que a resposta HTTP recebida foi 201, indicando que os registros foram criados com sucesso no servidor central. Os logs apresentaram mensagens como:

```
Inventário 10.0.10.1 VLAN 10 -> HTTP 201
Inventário 10.0.20.1 VLAN 20 -> HTTP 201
Inventário 10.0.40.40 VLAN 40 -> HTTP 201
Hosts encontrados: 15
Aguardando 60 segundos...
```

```

        "mac_address": "Desconhecido",
        "open_ports": [],
        "os_fingerprint": "Desconhecido",
        "services": [],
        "vlan": 60
    },
    {
        "hostname": "10.0.60.20",
        "ip_address": "10.0.60.20",
        "mac_address": "Desconhecido",
        "open_ports": [],
        "os_fingerprint": "Desconhecido",
        "services": [],
        "vlan": 60
    }
]
}
Inventário 10.0.10.1 VLAN 10 -> HTTP 201
Inventário 10.0.10.10 VLAN 10 -> HTTP 201
Inventário 10.0.20.1 VLAN 20 -> HTTP 201
Inventário 10.0.20.10 VLAN 20 -> HTTP 201
Inventário 10.0.30.1 VLAN 30 -> HTTP 201
Inventário 10.0.40.1 VLAN 40 -> HTTP 201
Inventário 10.0.40.10 VLAN 40 -> HTTP 201
Inventário 10.0.40.20 VLAN 40 -> HTTP 201
Inventário 10.0.40.30 VLAN 40 -> HTTP 201
Inventário 10.0.40.40 VLAN 40 -> HTTP 201
Inventário 10.0.50.1 VLAN 50 -> HTTP 201
Inventário 10.0.50.10 VLAN 50 -> HTTP 201
Inventário 10.0.60.1 VLAN 60 -> HTTP 201
Inventário 10.0.60.10 VLAN 60 -> HTTP 201
Inventário 10.0.60.20 VLAN 60 -> HTTP 201
Hosts encontrados: 15

Aguardando 60 segundos...
Pressione CTRL+C para encerrar o scanner.

```

Figura 14 — Final do ciclo de scan, mostrando envio dos inventários para a VM1 com HTTP 201 e total de 15 hosts encontrados.

## 6.1 Interpretação da saída do scanner

A saída exibida nos logs do container é uma parte importante da validação do projeto. Ela permite acompanhar em tempo real cada etapa executada pelo scanner. Quando aparece a mensagem "Novo scan iniciado", o programa está começando um novo ciclo de monitoramento. Em seguida, as mensagens de descoberta indicam quais redes e VLANs estão sendo percorridas.

```

Novo scan iniciado
Descobrimos hosts na rede 10.0.10.0/24 VLAN 10...

```

```

Descobrindo hosts na rede 10.0.20.0/24 VLAN 20...
Descobrindo hosts na rede 10.0.30.0/24 VLAN 30...
Descobrindo hosts na rede 10.0.40.0/24 VLAN 40...
Descobrindo hosts na rede 10.0.50.0/24 VLAN 50...
Descobrindo hosts na rede 10.0.60.0/24 VLAN 60...

```

Depois da descoberta inicial, o scanner passa a escanear cada IP encontrado. Nessa etapa são executadas as funções de descoberta de portas, serviços, sistema operacional, MAC e hostname. O inventário JSON impresso nos logs mostra o resultado estruturado dessa coleta.

```

{
  "hostname": "vm3-descoberta",
  "ip_address": "10.0.40.40",
  "mac_address": "fa:16:3e:9d:43:e2",
  "open_ports": [22],
  "os_fingerprint": "Linux 2.6.X|5.X",
  "services": ["ssh"],
  "vlan": 40
}

```

Após montar o inventário, o scanner envia os dados para a VM1. As mensagens com HTTP 201 indicam que o servidor central recebeu o dado e criou o registro com sucesso. Por fim, a mensagem "Hosts encontrados: 15" apresenta o total de hosts identificados no ciclo, e "Aguardando 60 segundos" indica que o programa entrou no intervalo antes da próxima varredura.

```

Inventário 10.0.40.40 VLAN 40 -> HTTP 201
Hosts encontrados: 15
Aguardando 60 segundos...

```

Essa saída comprova o funcionamento completo do fluxo: varredura das VLANs, geração do inventário, envio para a VM1, armazenamento do estado e repetição automática do ciclo.

## 7 REPOSITÓRIO GIT E DOCUMENTAÇÃO

O projeto foi versionado em um repositório Git, atendendo ao requisito de entrega de scripts, códigos, documentação e arquivos de configuração. O repositório contém o código principal do scanner, o Dockerfile, o arquivo Docker Compose, o arquivo de comandos e demais arquivos relacionados ao projeto.

O repositório pode ser clonado com o comando `gh repo clone DanielPMarchesi/vm3-discovery`. A utilização do Git foi importante para manter o histórico de desenvolvimento e permitir que o projeto fosse acessado por outras pessoas. Além do

código, foi criado o arquivo `COMANDOS.md` contendo instruções para baixar a imagem Docker e executar o projeto com Docker Compose.

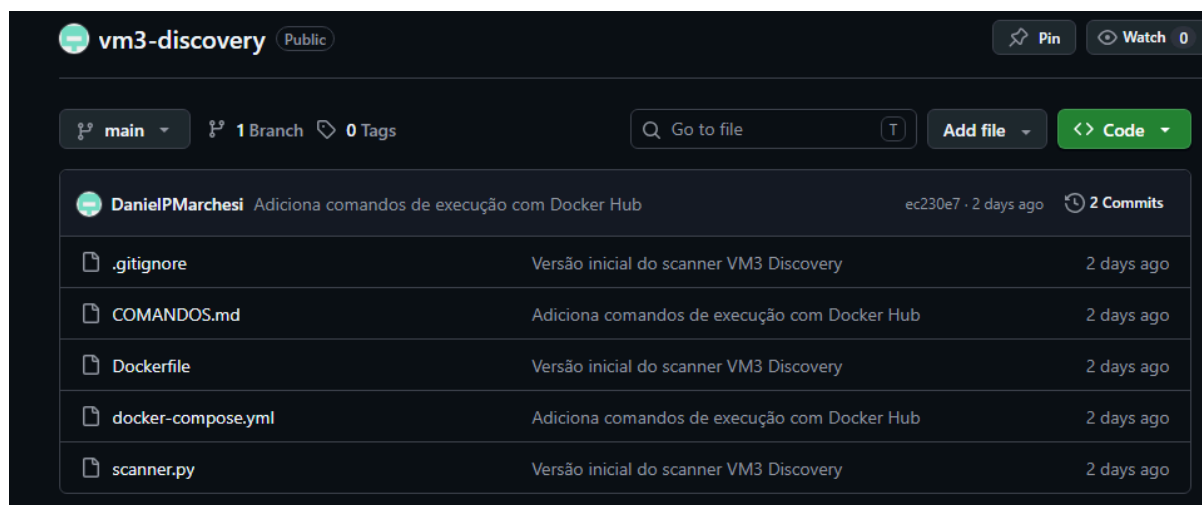


Figura 15 — Repositório GitHub do projeto VM3 Discovery, contendo scripts, código, documentação e arquivos de configuração.

## 8 DECISÕES TÉCNICAS E JUSTIFICATIVAS

Durante o desenvolvimento do projeto, algumas decisões técnicas foram tomadas para garantir o funcionamento do scanner. A primeira foi utilizar o Nmap como ferramenta principal de descoberta, por ser uma ferramenta consolidada capaz de realizar descoberta de hosts, varredura de portas, identificação de serviços e fingerprint de sistema operacional.

Outra decisão importante foi utilizar JSON como formato de inventário. O JSON é simples, organizado e compatível com APIs REST, o que facilitou a integração entre VM3 e VM1. O scanner foi configurado para execução contínua com intervalo de 60 segundos, permitindo que a VM3 monitore mudanças na rede sem depender de execução manual.

O armazenamento do estado anterior em arquivo local foi adotado por simplicidade, sendo suficiente para comparar o estado atual com o anterior sem necessidade de banco de dados próprio. A utilização de Docker padronizou o ambiente, reduzindo problemas de dependências e facilitando a execução por outros usuários.

A opção `network_mode: host` foi necessária para que o scanner acesse diretamente a rede da VM. A opção `privileged: true` foi utilizada para permitir que o Nmap execute operações de rede com as permissões adequadas. Por fim, campos que não puderam ser identificados, como MAC address ou sistema operacional, foram registrados como "Desconhecido", evitando campos vazios e deixando explícito que a informação não foi obtida.

## 9 LIMITAÇÕES ENCONTRADAS

Apesar de o sistema ter cumprido os objetivos principais do projeto, algumas limitações foram identificadas durante a implementação e os testes. Essas limitações estão relacionadas ao funcionamento real das redes, às permissões disponíveis no ambiente e ao prazo de desenvolvimento. Elas não invalidam o scanner, mas ajudam a explicar por que alguns campos do inventário aparecem como Desconhecido ou como o próprio endereço IP.

A principal limitação ocorreu na identificação do endereço MAC de hosts localizados em outras VLANs. O scanner utiliza a tabela de vizinhança da própria VM3, acessada pelo comando `ip neigh`, para tentar obter o MAC dos hosts encontrados. Essa técnica funciona melhor para dispositivos da mesma VLAN, pois o ARP opera na camada de enlace e fica restrito ao mesmo domínio de broadcast. Como a VM3 está conectada diretamente à VLAN 40, ela consegue obter com mais facilidade os MACs dessa VLAN.

Para hosts em outras VLANs, a comunicação ocorre por meio do gateway. Nesse cenário, a VM3 não faz ARP diretamente para o host remoto; ela resolve apenas o MAC do próximo salto, ou seja, o gateway da sua própria VLAN. O roteador encaminha pacotes IP entre redes, mas não encaminha broadcasts ARP entre VLANs. Por isso, a VM3 não consegue enxergar diretamente o MAC real de hosts nas VLANs 10, 20, 30, 50 e 60.

Foi analisada a possibilidade de obter esses MACs consultando automaticamente o gateway, pois o gateway possui interfaces nas diferentes VLANs e poderia ter informações da tabela ARP dessas redes. Porém, essa solução não foi finalizada dentro do prazo do projeto. Ela dependeria de autenticação SSH automática, chaves de acesso disponíveis dentro do container Docker, permissões no gateway, cliente SSH instalado no container e tratamento específico da saída dos comandos do roteador. Além disso, os endereços .1 das VLANs são interfaces locais do próprio gateway e não aparecem como vizinhos na tabela `ip neigh`. Por esse motivo, a solução final manteve o comportamento mais seguro: MAC identificado quando disponível localmente e Desconhecido quando não foi possível obter a informação de forma confiável.

Outra limitação ocorreu na identificação do hostname. O scanner tenta obter o nome do host utilizando resolução reversa de DNS por meio da função `socket.gethostbyaddr(ip)`. Essa técnica depende da existência de registros PTR no DNS do ambiente. Quando esse registro existe, o scanner consegue apresentar um nome, como ocorreu com a própria VM3. Quando não existe DNS reverso configurado para determinado IP, o Python não consegue resolver o nome e o scanner utiliza o próprio endereço IP como identificação.

A correção ideal para esse ponto seria configurar adequadamente o DNS reverso das redes monitoradas, criando zonas reversas para as VLANs e registros PTR para os hosts. Essa

configuração permitiria que consultas do tipo IP para nome funcionassem de forma padronizada. No entanto, essa parte não foi concluída no prazo do trabalho, pois exigiria ajustes no serviço de DNS do ambiente e validação para todas as VLANs. Por isso, no inventário final, alguns hosts permanecem com o campo hostname igual ao endereço IP.

Também houve limitação na identificação do sistema operacional. O fingerprint do Nmap não é uma identificação absoluta; ele depende das respostas que o host envia aos probes TCP, UDP e ICMP. Quando um host possui firewall, filtra pacotes, responde poucos testes, não possui portas abertas suficientes ou não apresenta portas fechadas para comparação, o Nmap pode não conseguir estimar o sistema operacional com confiança. Nesses casos, o campo `os_fingerprint` é registrado como Desconhecido.

Além disso, alguns serviços podem aparecer com identificação aproximada ou incompleta, pois a detecção de serviço depende de banners, respostas específicas e assinaturas conhecidas pelo Nmap. Quando o serviço não responde de forma clara, o Nmap pode classificar de maneira genérica ou incerta.

Por fim, também foi observada uma limitação relacionada à persistência de rotas. Algumas rotas adicionadas manualmente com `ip route replace` funcionam durante a sessão atual da VM, mas podem ser perdidas após reinicialização caso não sejam configuradas permanentemente no Netplan. Em uma versão futura, as rotas específicas para manter o SSH pela interface externa poderiam ser persistidas diretamente na configuração de rede.

Como melhorias futuras, recomenda-se concluir a integração para obtenção de MACs por fontes que enxerguem as demais VLANs, como gateway, OpenStack ou SNMP, configurar DNS reverso para os hostnames, persistir todas as rotas no Netplan e aprimorar o tratamento das respostas do Nmap. Essas melhorias não foram finalizadas no prazo, mas foram identificadas e documentadas como evolução natural do projeto.

## **10 USO DE INTELIGÊNCIA ARTIFICIAL**

Durante o desenvolvimento do projeto, foi utilizada IA generativa como ferramenta de apoio. A IA auxiliou na compreensão de conceitos de redes, interpretação de comandos Linux, explicação do funcionamento do Nmap, estruturação do código Python, uso do Docker e criação de documentação.

A IA foi utilizada como apoio ao aprendizado e à organização das ideias. Todas as configurações, testes e validações foram realizados no ambiente real da VM3. Portanto, a IA não substituiu a execução prática do projeto, mas auxiliou na documentação, na análise dos erros e na explicação técnica das soluções adotadas.

## 11 DEMONSTRAÇÃO FUNCIONAL

A demonstração funcional do projeto pode ser realizada diretamente na VM3. Inicialmente, é possível mostrar a configuração das interfaces com `ip a`, comprovando que a VM possui acesso pela rede externa e pela VLAN interna. Em seguida, a tabela de rotas pode ser exibida com `ip route`, demonstrando que a rota padrão está configurada pela `ens4` e que a rota do SSH permanece pela `ens3`.

Depois, acessa-se a pasta do projeto e executa-se o container com Docker Compose:

```
cd ~/vm3-discovery
docker compose up -d
docker ps
docker logs -f vm3-discovery
```

Durante a demonstração, os logs mostram o scanner descobrindo hosts nas VLANs, gerando o inventário JSON e enviando os dados para a VM1. O retorno HTTP 201 confirma que os dados foram recebidos pelo servidor central.

## 12 CONCLUSÃO

O projeto VM3 Discovery atingiu o objetivo proposto. Foi desenvolvida uma máquina virtual capaz de atuar como agente de descoberta e monitoramento de rede, configurada com duas interfaces, permitindo acesso externo pela `ens3` e comunicação com as redes internas pela `ens4`.

O scanner desenvolvido em Python utilizou o Nmap para descobrir hosts ativos, identificar portas abertas, serviços e sistemas operacionais. Os dados coletados foram organizados em formato JSON e enviados para a VM1 por meio de requisições HTTP.

Os testes realizados demonstraram que o scanner foi capaz de percorrer múltiplas VLANs, identificar hosts, gerar inventários e enviar os resultados ao servidor central com sucesso. A utilização de Docker e Docker Compose facilitou a execução e tornou o projeto mais simples de replicar.

Apesar das limitações naturais encontradas, como a dificuldade de identificar MAC address em outras VLANs, a ausência de DNS reverso para alguns hostnames e a possibilidade de imprecisão no fingerprint de sistema operacional, o sistema cumpriu sua função dentro do escopo acadêmico. Essas limitações foram documentadas e tratadas como pontos de melhoria futura, deixando claro que o projeto é funcional, mas ainda pode evoluir em integração com DNS, gateway, OpenStack e mecanismos mais completos de descoberta.

## REFERÊNCIAS

- NMAP.ORG.** *Nmap: the Network Mapper*. Disponível em: <https://nmap.org>. Acesso em: 2026.
- DOCKER INC.** *Docker Documentation*. Disponível em: <https://docs.docker.com>. Acesso em: 2026.
- DOCKER INC.** *Docker Compose Documentation*. Disponível em: <https://docs.docker.com/compose>. Acesso em: 2025.
- MARCHESI, Daniel Prezotti.** *Repositório VM3 Discovery*. Disponível em: <https://github.com/DanielPMarchesi/vm3-discovery>. Acesso em: 2026.